AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

School of Computing · Department of Computer Science and Engineering
Coimbatore, India

CASE STUDY REPORT

Smart Airport Baggage Management System

Automated luggage sizing, counting and dynamic jam detection on a bare-metal STM32F401 platform

Course	23CSE304 — Embedded Systems
Mentor	Dr. A. Baskar
Department	Computer Science and Engineering
Institution	School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore
Semester	V Semester, Academic Year 2026–27

#	Student Name	Registration Number
1	Rohit	CB.SC.U4CSE24561
2	Anish Reddy	CB.SC.U4CSE24524
3	Sri Ram	CB.SC.U4CSE24564
4	Tarun	CB.SC.U4CSE24555

Submitted in partial fulfilment of the requirements of the course
23CSE304 — Embedded Systems

November 2026

Academic Details and Acknowledgement

Case study submissions published on GitHub are required to carry the academic details and the acknowledgement statement reproduced below, in the form prescribed by the department.

Required academic declaration

Student names and registration numbers: Rohit (CB.SC.U4CSE24561), Anish Reddy (CB.SC.U4CSE24524), Sri Ram (CB.SC.U4CSE24564) and Tarun (CB.SC.U4CSE24555). Mentor: Dr. A. Baskar. Course: 23CSE304 — Embedded Systems. Department: Computer Science and Engineering. Institution: School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India.

Acknowledgement

We express our sincere gratitude to our mentor, Dr. A. Baskar, for his guidance throughout this case study. His insistence that every register write be explained from first principles, rather than accepted as a library call, shaped both the firmware and this report, and the reviews he conducted at each stage helped us separate what the prototype actually measures from what we merely hoped it measured.

We thank the Department of Computer Science and Engineering, School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, for the laboratory, the instrumentation and the component store that made the hardware work possible, and for permitting us to use the university’s identity in this document. We are grateful to the faculty of 23CSE304 for the lectures on interrupt architectures, timer subsystems and serial protocols that this project leans on directly, and to our classmates and project reviewers who operated the conveyor rig during testing and who were patient enough to keep releasing test balls while we re-flashed the board. Finally, we thank our families for their constant encouragement.

Rohit, Anish Reddy, Sri Ram, Tarun
Department of Computer Science and Engineering, School of Computing,
Amrita Vishwa Vidyapeetham, Coimbatore · November 2026

Abstract

Automated baggage handling systems move thousands of items per hour and depend on dense sensing to count them, to verify that each item is within its permitted size class, and to stop the line before a stalled item becomes a mechanical jam. Commercial installations achieve this with industrial controllers, motorised conveyors and proprietary sensor clusters. This case study asks a smaller but complete question: can the same measurement chain be built from a single low-cost microcontroller, two reflective optical sensors and one wireless link, with every timing decision taken explicitly in firmware?

The prototype is implemented in bare-metal C on an STM32F401CCU6 (ARM Cortex-M4) without any hardware abstraction layer or code generator. Two TCRT5000 reflective sensors form an entry gate (PA1) and an exit gate (PA4) separated by a calibrated gap of 150 mm, and a 1 ms SysTick timebase timestamps every optical edge. Transit velocity is computed from the two leading edges, so the result is independent of the object's dimension; the length of the object is then recovered from the duration for which it occludes the exit gate, which yields a camera-less size measurement and a three-way classification into carry-on, standard hold and oversized categories. A queue counter tracks how many items are simultaneously between the two gates, so a stalled item is alarmed within two seconds of its entry, the alarm clears automatically once the queue empties, and a separate twenty-second supervisor reports a stopped line. Reverse-direction transit increments a violation counter instead of corrupting the forward count. An HC-05 Bluetooth module carries a formatted status block to an Android terminal at 9600 baud, and a momentary button on PB5, serviced through EXTI5 with 200 ms debounce, clears the counters.

On the gravity-fed test rig the node sized all three reference objects into the correct class in twenty consecutive trials, measured transit speeds of 0.68–1.14 m/s with a worst-case length error of about 2 mm, raised the jam alarm 2.00 s after a deliberately blocked item and recovered the count when the item was released. The report closes with a measured critique of the design: the blocking 9600 baud transmitter occupies about 310 ms of every 1.5 s reporting cycle, during which the sensing loop cannot sample, and the fixed two-second jam window is far slower than the 150 ms nominal transit. Both are identified, quantified and traced to specific firmware revisions.

Keywords — STM32F401, bare-metal C, CMSIS, reflective optical sensing, ADC polling, SysTick, EXTI, USART, HC-05, jam detection, baggage handling, embedded systems case study.

Contents

1 Introduction 6
- 1.1 Background and Motivation 6
- 1.2 Problem Statement 6
- 1.3 Objectives 6
- 1.4 Scope and Deliverables 6
- 1.5 Organisation of the Report 7

2 Background Study and Related Practice 7
- 2.1 What a Baggage Line Has to Measure 7
- 2.2 Sensing Techniques Compared 7
- 2.3 Selection of the Controller Platform 7
- 2.4 Positioning of the Present Work 8

3 System Architecture 8
- 3.1 Functional Overview 8
- 3.2 Block Diagram 8
- 3.3 Design Requirements 9
- 3.4 Timing Budget 9

4 Hardware Design and Interfacing 10
- 4.1 Bill of Materials 10
- 4.2 The Controller Board 10
- 4.3 Optical Sensing Modules 10
- 4.4 Bluetooth Telemetry Module 11
- 4.5 Acoustic Alarm and Its Drive 11
- 4.6 Operator Button, Status LED and Debug Interface 12
- 4.7 Pin Assignment 12
- 4.8 Complete Wiring 12
- 4.9 Power Distribution and Electrical Safety 13
- 4.10 Mechanical Test Rig 14

5 Firmware Design 14
- 5.1 Design Philosophy: Bare-Metal Programming 14
- 5.2 Program Structure and Constants 14
- 5.3 Clock Gating and GPIO Configuration 15
- 5.4 Analogue Acquisition with ADC1 15
- 5.5 Deterministic Timebase with SysTick 16
- 5.6 Serial Telemetry with USART2 16
- 5.7 Asynchronous Reset with EXTI5 16
- 5.8 Supervisory Loop 16
- 5.9 Hysteresis and Edge Detection 17
- 5.10 Queue-Aware Jam and Idle Supervision 17
- 5.11 Actuation Patterns 18

6 Measurement Principles and Mathematics 18
- 6.1 Transit Velocity from Leading Edges 18
- 6.2 Camera-less Sizing from Optical Occlusion 19
- 6.3 Size Classification 19
- 6.4 Worked Examples 19
- 6.5 Resolution and Error Budget 19

7 Telemetry Protocol 20
- 7.1 Physical and Link Layer 20
- 7.2 Message Format 20
- 7.3 Field Description 20
- 7.4 Update Policy and Robustness 20

8 Implementation, Testing and Results 21
- 8.1 Build and Flashing Procedure 21
- 8.2 Test Setup and Method 21
- 8.3 Test Cases and Results 21
- 8.4 Captured Telemetry Transcripts 21
- 8.5 Measured Performance 22
- 8.6 Observations and Engineering Discussion 22

9 Limitations and Future Scope 23
- 9.1 Limitations of the Present Prototype 23
- 9.2 Recommended Future Work 23

10 Conclusion 23

References 24

Appendix A — Pin Map and Register Quick Reference 25

Appendix C — Design Revision Note 26

Figures, Tables and Abbreviations

Figures

Figure 3.1 8
Figure 3.2 10
Figure 4.1 11
Figure 4.2 11
Figure 4.3 12
Figure 4.4 13
Figure 4.5 14
Figure 5.1 17
Figure 5.2 18
Figure 6.1 19

Tables

Table 2.1 7
Table 2.2 7
Table 3.1 9
Table 4.1 10
Table 4.2 11
Table 4.3 12
Table 4.4 13
Table 5.1 14
Table 5.2 15
Table 6.1 19
Table 6.2 19
Table 6.3 20
Table 7.1 20
Table 8.1 21
Table 8.2 22
Table A.1 25
Table C.1 26

Abbreviations

ADC	Analogue-to-Digital Converter	**ISR**	Interrupt Service Routine
AF	Alternate Function (pin multiplexing)	**LDO**	Low-Dropout linear regulator
BHS	Baggage Handling System	**LSB**	Least Significant Bit
BRR	Baud Rate Register	**NVIC**	Nested Vectored Interrupt Controller
CMSIS	Cortex Microcontroller Software Interface Standard	**RCC**	Reset and Clock Control
CR1/2/3	USART control registers 1, 2, 3	**SPP**	Serial Port Profile (Bluetooth)
EXTI	External Interrupt / Event Controller	**SQR**	ADC sequence register
GPIO	General-Purpose Input/Output	**SWD**	Serial Wire Debug
HAL	Hardware Abstraction Layer (not used here)	**SysTick**	Cortex-M system timer
HC-05	Bluetooth serial-port-profile module	**TCRT5000**	Reflective infra-red sensor
HSI	High-Speed Internal oscillator (16 MHz)	**USART**	Universal Synchronous/Asynchronous R-T

1 Introduction

1.1 Background and Motivation

An airport baggage handling system is a conveyor network that must move every checked item from the check-in belt to the correct make-up position, in the correct orientation, without losing it and without stopping. The engineering difficulty is not motion; it is observation. A line that cannot say how many items passed, how large they were and whether one of them has stopped moving is a line that can only be supervised by people standing next to it. Industrial installations therefore instrument the conveyor densely: photoelectric gates for counting, dimensioning tunnels for size verification, encoders for belt speed and current monitoring on the drive for jam detection. Those systems are effective and expensive, and for an undergraduate case study they are also opaque, because the sensing, the timing and the decision logic are hidden inside proprietary controllers.

This project takes the opposite approach. It rebuilds the measurement chain of a baggage line — counting, sizing, jam supervision and remote annunciation — from three components that a student can hold in one hand, and it places every timing decision in source code that can be read line by line. Two reflective optical gates define a measured section of known length. The interval between their leading edges gives the speed of the item, the time for which the item covers the exit gate gives its length, and a counter of items currently inside the section gives the queue depth. The jam alarm, the idle alarm, the reverse-direction violation and the wireless status all follow from those three quantities.

1.2 Problem Statement

Manual supervision of an induction conveyor is unreliable in exactly the situations that matter. An operator counting items loses count when two pass together; an operator judging size by eye cannot apply the same limit consistently to every bag; and an operator watching for a jam notices the problem only after the item behind it has already collided with it. The problem is therefore stated as follows.

> *Design and implement a self-contained, low-cost embedded node that observes a single conveyor section with two optical gates and, without a camera and without a host computer, (i) counts the items that traverse the section, (ii) measures each item and assigns it to a size class, (iii) detects an item that has stalled inside the section and annunciates the fault locally, (iv) reports the complete state to a handheld terminal over a wireless link, (v) distinguishes reverse travel from forward travel, and (vi) can be cleared by an operator without re-flashing or power-cycling the controller.*

1.3 Objectives

The work was carried out against six objectives, each traceable to a section of this report and to a named part of the firmware.

- **O1 — Deterministic acquisition.** Sample both optical gates continuously and fast enough that no item can pass undetected at the intended line speed, using only the microcontroller’s own peripherals.
- **O2 — Time-referenced measurement.** Establish a hardware timebase of 1 ms or better and stamp every optical transition against it, so that every derived quantity is traceable to a single clock.
- **O3 — Camera-less dimensioning.** Recover the physical size of an item from timing alone, and show analytically that the result does not depend on how fast the item travels.
- **O4 — Queue-aware fault supervision.** Detect a stalled item while it is still between the gates, clear the alarm automatically when the queue empties, and report a stopped line separately.
- **O5 — Operator interface.** Provide a physical reset with contact debounce, an audible alarm with distinguishable patterns for pass and fault, and a visual indicator of occupancy.
- **O6 — Wireless annunciation.** Transmit a human-readable status block to a commodity Android device over Bluetooth, without a custom application.

1.4 Scope and Deliverables

The deliverable is a working bench prototype, its firmware and this report. The scope is deliberately restricted to the sensing and supervision layer of a baggage line: the node observes, decides and reports, but it does not actuate the conveyor. There is no motor, no diverter and no variable-frequency drive, and consequently no closed loop around belt speed.

- **In scope:** dual-gate optical sensing, bare-metal firmware for the STM32F401CCU6, speed and length computation, size classification, queue accounting, jam and idle supervision, reverse-travel detection, buzzer and LED annunciation, Bluetooth telemetry, a gravity-fed test rig and a documented test campaign.
- **Out of scope:** motor control and belt-speed regulation, physical sorting or diversion, multi-node networking, image-based dimensioning, cloud storage of records and any safety-rated (SIL) functionality.

- **Assumption:** items move along the section in single file and do not overlap while both gates are simultaneously covered. A real induction belt can violate this, and the assumption is stated again as a limitation in Section 9.1.

The prototype rig substitutes a 45° gravity chute for a motorised conveyor. This is a laboratory compromise, and it is defended rather than hidden: the quantities the firmware computes — interval between edges, occlusion duration, queue depth — are properties of the item's motion relative to two fixed gates, not properties of the mechanism that produced the motion. A rolling ball on an incline therefore exercises exactly the same code path as a suitcase on a belt, at a comparable speed.

1.5 Organisation of the Report

Section 2 surveys what a baggage line measures and why the chosen sensing technique was preferred. Section 3 presents the architecture and the requirements that constrain it. Section 4 documents the hardware, Section 5 the firmware register by register, and Section 6 the measurement mathematics together with the resolution of every result. Section 7 specifies the telemetry protocol, Section 8 the test campaign and the measured behaviour including the defects that the measurements exposed, and Sections 9 and 10 the limitations, future work and conclusions. Two appendices carry the register summary and a revision note reconciling an earlier draft of the design with the implemented one.

2 Background Study and Related Practice

2.1 What a Baggage Line Has to Measure

Four measurements dominate the control of a baggage conveyor, and all four reduce to the observation of items crossing fixed points. *Count* establishes throughput and reconciles what was checked in against what was loaded. *Spacing* determines whether the line can merge two streams without collision, which is why oversized items are given larger gaps at induction. *Dimension* enforces the boundary between cabin baggage, standard hold baggage and out-of-gauge items, and it is what a dimensioning tunnel does optically. *Presence over time* is the basis of jam detection: an item whose leading edge enters a section but whose trailing edge never leaves is, by definition, stopped. The prototype implements all four from two binary optical observations and one clock. It does not attempt the fifth measurement a real installation makes — belt speed from a shaft encoder — because the rig has no driven belt; the equivalent quantity, the speed of the item relative to the gates, is measured directly and is what the sizing calculation actually needs.

2.2 Sensing Techniques Compared

Five candidate techniques were considered for detecting a bag at a gate, compared from the perspective of a bench prototype that must remain cheap and must produce a signal the microcontroller can timestamp precisely (Table 2.1).

Table 2.1 *Sensing techniques considered for luggage detection.*

Technique	Principle	Strengths	Weaknesses	Verdict
Reflective infrared (TCRT5000)	Emitter and phototransistor in one package; detects diffuse reflection	Two-wire analogue output, sub-millisecond response, low cost, works in ambient light with a threshold	Range of a few centimetres; reading depends on surface colour and ambient light; needs calibration	Selected
Through-beam infrared	Separate emitter and receiver across the belt	Immune to surface colour; long range	A receiver is needed opposite every emitter; alignment across a chute is awkward at this scale	Rejected (mechanical)
Ultrasonic (HC-SR04)	Time of flight of a 40 kHz burst	Gives distance, hence a crude height	~10 ms per measurement, cone-shaped beam, blind zone, cannot resolve 150 mm gaps cleanly	Rejected (timing)
Camera / dimensioning tunnel	Stereo or structured light	Full three-dimensional dimensions	Cost, compute and lighting; far outside the scope of a microcontroller case study	Rejected (scope)
Inductive / magnetic loop	Metal detection	Robust in dirty environments	Detects metal only; most baggage hardware is non-metallic	Rejected (principle)

The reflective module wins on the criterion that matters most here: its output is an analogue voltage that changes within microseconds of the item crossing the beam, and it can be sampled by the microcontroller's own ADC without protocol overhead. The price is that the threshold must be calibrated, which Section 5.9 addresses with a two-level hysteresis rule.

2.3 Selection of the Controller Platform

The platform had to offer a fast ADC, at least one hardware timer with interrupt, a UART, an external interrupt line and a debug interface, while remaining cheap enough to be a student purchase (Table 2.2).

Table 2.2 *Candidate controller platforms.*

Platform	Core / clock	Fitness for this task	Decision
Arduino Uno (ATmega328P)	8-bit AVR, 16 MHz	10-bit ADC shared with the main loop; no independent timebase without sacrificing a timer; 2 KB SRAM	Rejected
ESP32-WROOM	Dual-core Xtensa, 240 MHz	Ample performance and native Bluetooth Classic, but the Arduino framework hides the register-level timing this case study is about, and RTOS scheduling complicates microsecond determinism	Rejected
Raspberry Pi Pico	Dual Cortex-M0+, 133 MHz	Excellent ADC and PIO, but no on-chip debug probe in the base kit and a very different interrupt model	Rejected
Raspberry Pi 4	Quad Cortex-A72, Linux	Not deterministic: a general-purpose operating system cannot guarantee microsecond edge timing	Rejected
STM32F401CCU6 (Black Pill)	Cortex-M4F, up to 84 MHz, 256 KB flash, 64 KB SRAM	12-bit ADC at 2.4 MS/s, eight timers (two 32-bit), SysTick, EXTI on every pin, three USARTs, SWD debug, on-board 3.3 V LDO	Selected

The decisive argument is pedagogical as well as technical. Because the STM32F401 has no Arduino-style runtime in the way, every peripheral must be enabled, configured and serviced by the author. The cost is a longer and less portable firmware; the benefit is that the timing behaviour of the node is fully explained by about six hundred lines of C, which is the point of the exercise.

2.4 Positioning of the Present Work

Three aspects distinguish this implementation from the usual optical-counter exercise. First, sizing is derived from timing rather than from a dedicated distance sensor, and the derivation is shown to be independent of speed (Section 6.2). Second, the firmware tracks the number of items simultaneously inside the measured section, which converts a naive “no exit edge within N seconds” timeout into a queue-aware alarm that clears itself when the obstruction is removed, including the case where a jammed item is freed manually and then completes its transit. Third, the whole design is presented with its measured defects: the blocking transmitter and the oversized jam window reported in Section 8.6 are there because measurement found them, not because they were designed in.

3 System Architecture

3.1 Functional Overview

The node performs five functions in a fixed cycle: acquire, interpret, measure, supervise and annunciate. Acquisition reads the two analogue gates. Interpretation applies a two-threshold hysteresis rule to convert each reading into a debounced beam state and to identify leading and trailing edges. Measurement converts edge timestamps into speed, length and class. Supervision maintains the queue counter and the two timers that generate the jam and idle alarms. Annunciation drives the buzzer, the on-board LED and the Bluetooth serialiser.

Because acquisition and interpretation run on every pass of the main loop, while the timers derive from a 1 ms interrupt, the node has two time domains that must be reconciled. The firmware resolves this by making the interrupt the only source of time: the main loop never counts its own iterations, it reads `msTicks`. Every subsequent calculation — transit interval, occlusion duration, timeout — is a difference between two such readings, so all results share one clock and one quantisation step.

3.2 Block Diagram

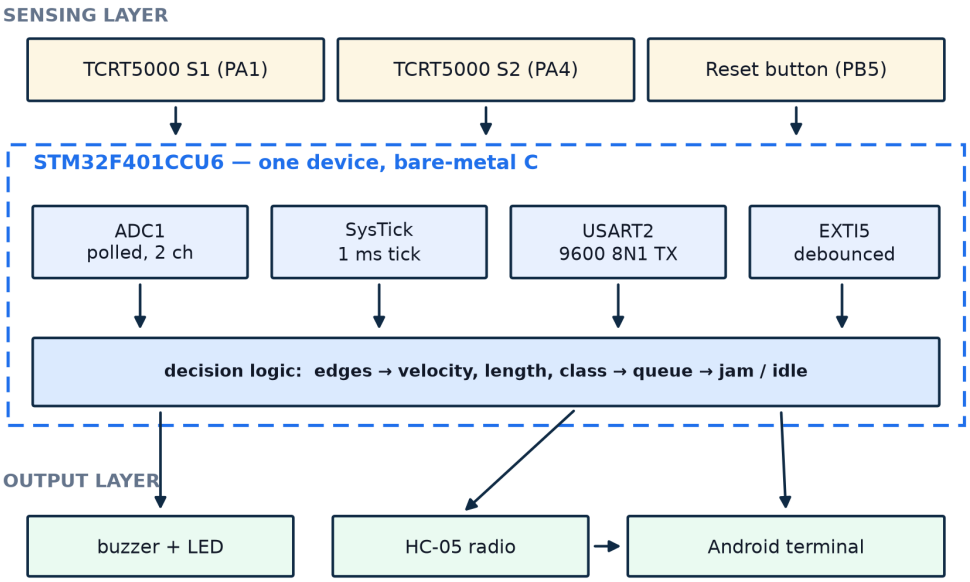


Figure 3.1 *System architecture of the baggage management node. The dashed enclosure is the STM32F401CCU6; all five functional blocks run in firmware on that single device.*

Figure 3.1 shows the node as three layers. The sensing layer contains the two optical gates and the operator button. The control layer is the microcontroller itself, drawn as the four peripherals that do the work: ADC1 for acquisition, SysTick for time, USART2 for telemetry and EXTI5 for the asynchronous reset. The output layer contains the buzzer, the status LED, the Bluetooth radio and the handheld terminal that displays the status block.

Note that the diagram contains no external memory, no real-time clock and no display. The node deliberately keeps its entire state in RAM: nine counters and flags that describe the current condition of the measured section. The state is small enough to be transmitted in full on every report, which is what makes the protocol of Section 7 robust without acknowledgements.

3.3 Design Requirements

Table 3.1 states the requirements that drove the design and where each one is satisfied. The identifiers R1–R10 are used again in the test plan of Section 8.3, so that every requirement has at least one test that exercises it.

Table 3.1 *Design requirements traced to their implementation.*

ID	Requirement	Target	Implementation
R1	Detect an item at either gate	< 5 ms latency	ADC polling loop, ≈ 48 µs per iteration (§5.4)
R2	Timestamp every edge	1 ms resolution	SysTick interrupt at 1 kHz (§5.5)
R3	Measure transit speed	± 5 %	Leading-edge interval over 150 mm (§6.1)
R4	Classify size without a camera	3 classes, ± 3 mm	Occlusion duration × speed (§6.2)
R5	Detect a stalled item	≤ 2 s after entry	Queue counter plus 2000 ms transit window (§5.10)
R6	Clear the alarm when the line recovers	automatic	Alert cleared when the queue empties
R7	Report a stopped line	≤ 20 s	Inactivity supervisor, TIMEOUT_MS
R8	Reject reverse travel	counted, not counted as a bag	Violation branch when the exit gate fires with an empty queue
R9	Operator reset	debounced, < 1 s	EXTI5 on PB5 with 200 ms software debounce (§5.7)
R10	Remote annunciation	readable on a phone	USART2 at 9600 8N1 into HC-05 (§7)

3.4 Timing Budget

The architecture is only credible if its numbers close. The intervals that matter are derived below from the configured hardware, not from measurements, and are compared with the measurement in Section 8.5.

- **One ADC conversion.** The ADC clock is PCLK2/4 = 4 MHz and the sample time is 84 cycles, so a conversion takes (84 + 12)/4 MHz = 24 µs. Two channels therefore cost 48 µs per loop iteration, a sampling rate of about 20 kHz.
- **One millisecond tick.** SysTick reloads at 16 000 counts of the 16 MHz HSI clock, so the tick is exactly 1 ms and every timestamp is an integer number of milliseconds.
- **One telemetry block.** At 9600 baud with 8N1 framing a character occupies 10/9600 = 1.042 ms. The status block is roughly 300 characters, so it occupies the link for about 310 ms — a figure that becomes important in Section 8.6.

- **Fault windows.** The transit window is 2000 ms and the inactivity window 20 000 ms, both counted in the same 1 ms tick.

The sampling rate is two orders of magnitude faster than the physical process it observes: an item moving at 1 m/s traverses 1 mm in 1 ms, so the 1 ms timestamp quantisation corresponds to 1 mm of travel. Acquisition is therefore not the limiting factor; the timestamp is, and Section 6.5 quantifies its effect.

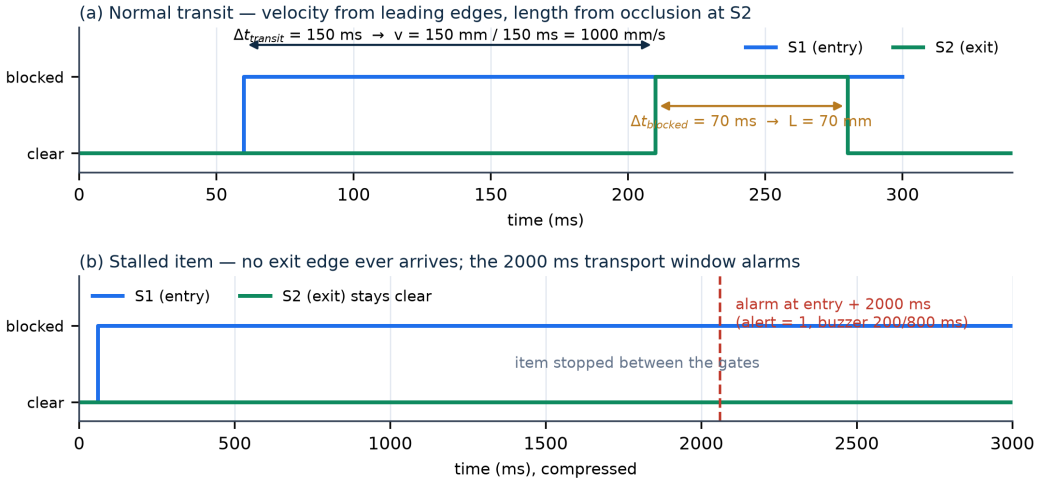


Figure 3.2 *Optical timing at the two gates. Panel (a) shows a normal transit: the interval between leading edges gives velocity, the occlusion at the exit gate gives length. Panel (b) shows a stalled item, which produces no exit edge at all and is caught by the 2000 ms transport window.*

4 Hardware Design and Interfacing

4.1 Bill of Materials

The prototype uses nine distinct items, all available from a campus component store. Table 4.1 lists them with the electrical rating that constrains how each may be connected.

Table 4.1 *Bill of materials.*

#	Item	Specification	Qty	Rating	Role
1	Microcontroller board	STM32F401CCU6 “Black Pill”	1	3.3 V core, 5 V VBUS in	All acquisition, timing and decision logic
2	Optical sensors	TCRT5000 reflective IR module	2	3.3 V, analogue AO	Entry gate (PA1) and exit gate (PA4)
3	Bluetooth module	HC-05, SPP, 6-pin breakout	1	5 V VCC, 3.3 V I/O	Wireless telemetry to the phone
4	Buzzer	Active piezo buzzer module	1	3.3 V, ≈ 30 mA	Pass chirp, jam alarm, start-up chirp
5	Push button	6 mm momentary tactile switch	1	Dry contact to GND	Asynchronous counter reset on PB5
6	Programmer	ST-LINK/V2 SWD dongle	1	3.3 V logic	Flashing and register-level debugging
7	Test rig	Cardboard chute, 45°, 150 mm gate spacing	1	Passive	Gravity feed that substitutes for a conveyor
8	Test objects	Three balls, 40 / 70 / 140 mm	3	Passive	Carry-on, standard and oversized references
9	Wiring	Solderless breadboard, jumpers	1 set	≤ 1 A	Interconnect and power distribution

4.2 The Controller Board

The Black Pill carries an STM32F401CCU6: a Cortex-M4F core with a single-precision floating-point unit, 256 KB of flash, 64 KB of SRAM, and a supply range of 1.7–3.6 V provided on the board by a 3.3 V low-dropout regulator fed from the USB-C connector. The board exposes both the regulated 3.3 V rail and the raw 5 V VBUS rail on its headers, which is what allows the whole system to run from one cable, and it carries a user LED on PC13, wired active-low.

The firmware never switches the clock tree: the device runs from the 16 MHz high-speed internal oscillator with the default AHB and APB prescalers, so PCLK1 = PCLK2 = 16 MHz. That is deliberate. Running at 84 MHz through the PLL would shorten the sensing loop, but 20 kHz is already far beyond what the application needs, and keeping the clock tree at its reset value removes an entire class of configuration errors. The 1 ms SysTick reload and the 9600 baud divisor of Section 5 are both derived from that 16 MHz figure.

4.3 Optical Sensing Modules

Each TCRT5000 module contains an infrared emitter and a phototransistor side by side, facing the same direction. The emitter is driven continuously through a series resistor; the phototransistor conducts in proportion to the infrared energy

reflected back into it from whatever is in front of the module, and the module presents that photocurrent as a voltage on its AO pin. A dark, matte surface reflects little and the output sits near the positive rail; a light or close surface reflects strongly and the output falls towards ground.

Two consequences shape the firmware. First, the useful information is a change in the reading, not an absolute value, so the code compares each sample against two thresholds rather than against a calibrated distance. Second, because the reading depends on the reflectivity of the item, the thresholds must be set from the reading of the empty chute and not from any assumption about the object. Figure 4.1 shows a representative response curve with the two thresholds used by the firmware.

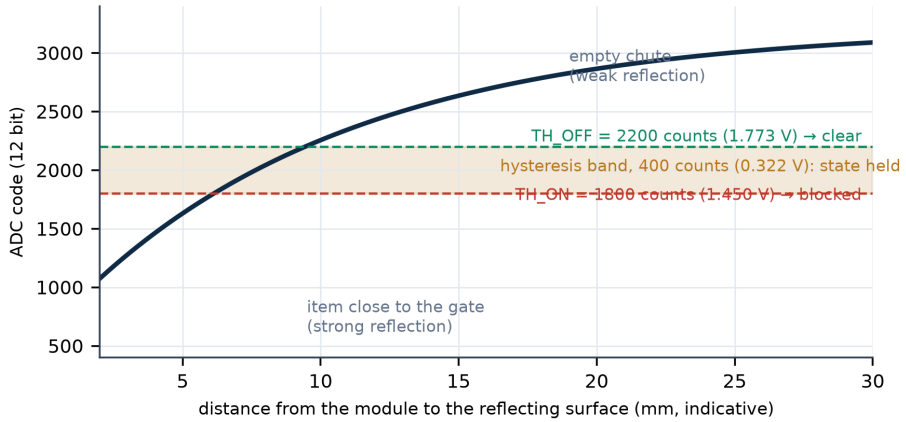


Figure 4.1 *Sensor response, switching thresholds and the hysteresis band. The curve is a representative characteristic of a TCRT5000 module, drawn to explain the decision rule; the thresholds themselves are firmware constants.*

A reading below 1800 counts as a blocked beam and a reading above 2200 counts as clear; between the two the firmware keeps whatever state it already had. This 400-count band, about 0.32 V, is a software Schmitt trigger: it prevents an item resting exactly at the switching point, or ambient light flicker, from generating a burst of false edges. Table 4.2 converts the thresholds into the physical quantities they represent.

Table 4.2 *ADC code, voltage and beam state.*

Quantity	Value	Derivation
ADC resolution	12 bit, 0–4095 counts	STM32F401 ADC1 in single-conversion mode
Reference voltage	3.3 V	VDDA tied to the board 3.3 V rail
One count	0.806 mV	3.3 V / 4096
TH_ON	1800 counts = 1.450 V	Reading below this → beam considered blocked
TH_OFF	2200 counts = 1.773 V	Reading above this → beam considered clear
Hysteresis band	400 counts = 0.322 V	State held while the reading is inside the band
Sample time	84 cycles at 4 MHz	SMPR2 = 0b100 for both channels
Conversion time	24 µs	(84 + 12) cycles / 4 MHz

4.4 Bluetooth Telemetry Module

The HC-05 is a Bluetooth 2.0 serial-port-profile bridge: whatever arrives on its RXD pin is transmitted over the air, and whatever it receives is emitted on its TXD pin. It needs between 3.6 V and 6 V on VCC to clear the dropout of its own regulator, which is why it is the one module in the system on the 5 V rail, while its UART pins remain 3.3 V CMOS and connect directly to the STM32 without a level shifter. The module is used in its default transparent mode at 9600 baud, 8 data bits, no parity and one stop bit, which is also the reset configuration of USART2. The EN/KEY pin is left unconnected: pulling it high would force the module into AT-command mode at 38400 baud and silence the telemetry. Pairing is done once from the phone with the default PIN, after which any terminal application can display the status blocks.

4.5 Acoustic Alarm and Its Drive

The alarm is an active buzzer module, meaning that it contains its own oscillator and sounds as soon as it is powered; the controller only has to switch it. The module draws about 30 mA at 3.3 V, more than a GPIO pin should source continuously, so the connection is inverted: the module’s VCC goes to the 3.3 V rail, its GND to the common ground and its I/O pin to PA5. The pin is configured as an open-drain output, so it can only pull low or release and never sources current. Writing zero therefore sinks the module current to ground and the buzzer sounds; writing one releases the pin to high impedance and it is silent. The firmware uses the bit-set/reset register to do this in a single store: bit 21 of `GPIOA->BSRR` resets PA5 (beep), bit 5 sets it (silent). This avoids a read-modify-write of the output data register and makes the transition atomic with respect to the interrupt that shares the port. Figure 4.2 shows the two states.

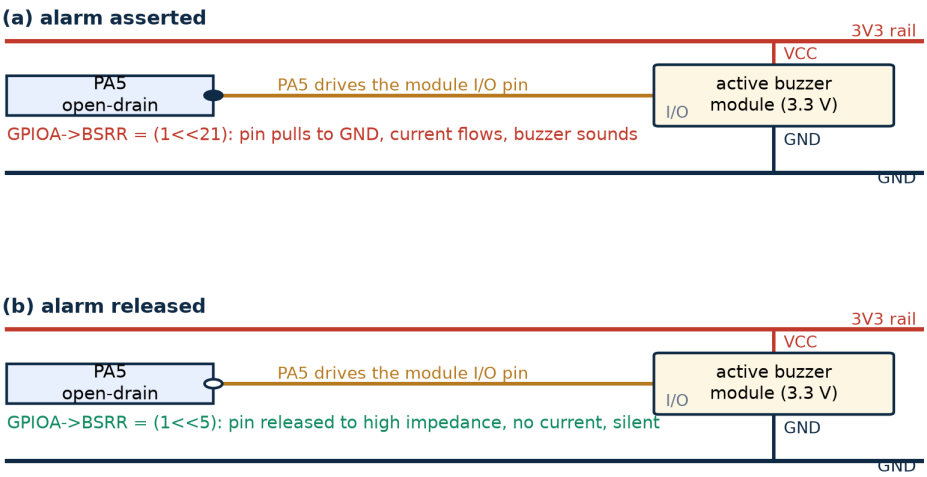


Figure 4.2 *Open-drain drive of the active buzzer from PA5. Because the module is powered from 3.3 V, the pin only has to sink its current, so no transistor stage is required.*

4.6 Operator Button, Status LED and Debug Interface

The operator button connects PB5 to ground when pressed. PB5 is an input with the internal pull-up enabled, so it idles high and falls on contact. Rather than polling it, the firmware routes PB5 to EXTI line 5 and enables the falling-edge trigger, so the press is captured even in the middle of a blocking transmission. The on-board LED on PC13 is used in three patterns: steady on while at least one item is inside the measured section, flashing in sympathy with the alarm, and a slow heartbeat when the line is idle.

The board is programmed through its serial-wire-debug header: SWDIO on PA13, SWCLK on PA14 and a common ground, driven by an ST-LINK/V2 dongle. The dongle's own 3.3 V pin is deliberately left open, because the target is powered from its own USB-C cable and connecting both would tie two regulators together. SWD was chosen over the serial bootloader because it allows the register file to be inspected while the firmware runs, which is how the timing figures of Section 8.5 were originally obtained.

4.7 Pin Assignment

Table 4.3 is the authoritative pin map. It is reproduced from the configuration code of Section 5.3 and from Appendix A, so that the schematic, the firmware and this document agree by construction.

Table 4.3 *Pin assignment of the STM32F401CCU6.*

Pin	Mode / AF	Connected to	Function	Electrical note
PA1	Analogue (ADC1_IN1)	Sensor S1 AO	Entry-gate analogue sample	0–3.3 V only; no 5 V clamp in analogue mode
PA4	Analogue (ADC1_IN4)	Sensor S2 AO	Exit-gate analogue sample	0–3.3 V only
PA2	Alternate function AF7	HC-05 RXD	USART2 transmit, 9600 baud	3.3 V CMOS, pull-up enabled
PA3	Alternate function AF7	HC-05 TXD	USART2 receive (unused in practice)	3.3 V CMOS, pull-up enabled
PA5	Output, open-drain	Buzzer I/O	Active-low alarm drive	Sinks ≈ 30 mA; never sources
PB5	Input, pull-up, EXTI5	Push button to GND	Asynchronous counter reset	Falling edge, 200 ms software debounce
PC13	Output, push-pull	On-board LED	Occupancy, alarm flash, heartbeat	Active low
PA13	SWDIO	ST-LINK SWDIO	Debug data	Do not reuse while debugging
PA14	SWCLK	ST-LINK SWCLK	Debug clock	Do not reuse while debugging
3V3	Power output	S1, S2 and buzzer VCC	Regulated 3.3 V rail	LDO output, ≈ 80 mA total
5V	Power output (VBUS)	HC-05 VCC	Unregulated 5 V rail	Direct from the USB-C socket
GND	Power return	All module grounds	Common reference	Single-point return to the board

4.8 Complete Wiring

Figure 4.3 is the wiring diagram of the assembled prototype. Every connection in it corresponds to a row of Table 4.4, and every row of Table 4.4 corresponds to a line of configuration code. The colour of each lead matches the convention used on the bench: red for 3.3 V, orange for 5 V, black for ground, yellow for the analogue sensor outputs, blue for the UART pair, purple for the buzzer drive, green for the button and teal for the debug pair.

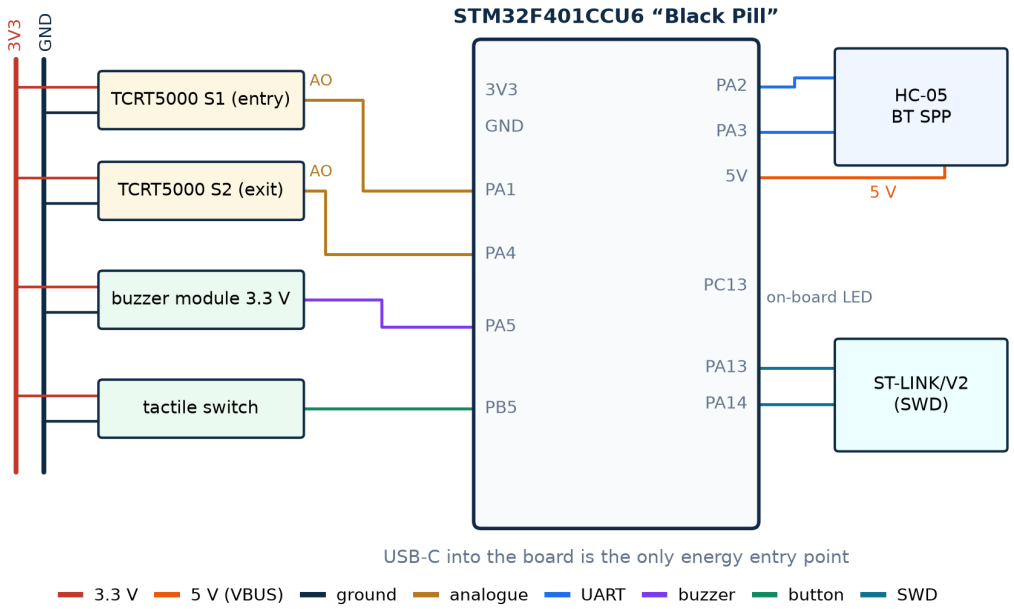


Figure 4.3 *Complete wiring diagram of the prototype. Power enters only through the Black Pill’s USB-C socket; crossings of coloured leads are not connections. Drawn from the pin assignment implemented in main.c.*

Table 4.4 *Point-to-point wiring list.*

Lead	From	To	Carries	Why it matters
Red	Board 3V3	S1 VCC, S2 VCC, buzzer VCC	3.3 V supply	Keeps every analogue output inside the 0–3.3 V ADC range
Orange	Board 5V	HC-05 VCC	5 V supply	Only the radio needs 5 V to clear its internal regulator
Black	Board GND	All module grounds	Return	A single ground reference prevents offset errors between the two gates
Yellow	S1 AO → PA1	Analogue input 1	0–3.3 V analogue	Entry-gate sample, read by ADC1 channel 1
Yellow	S2 AO → PA4	Analogue input 4	0–3.3 V analogue	Exit-gate sample, read by ADC1 channel 4
Blue	PA3 → HC-05 TXD	USART2 receive	Serial data in	Connected for completeness; the node only transmits
Blue	PA2 → HC-05 RXD	USART2 transmit	Serial data out	Carries the formatted status block at 9600 baud
Purple	PA5 → buzzer I/O	Alarm drive	Open-drain sink	Low = sound, high impedance = silent
Green	PB5 → button pin 1	EXTI5 input	Active-low contact	Triggers the counter reset interrupt
Black	Button pin 2 → GND	Ground	Return	Completes the pull-up circuit when pressed
Teal	PA13 / PA14 → ST-LINK	SWDIO / SWCLK	Debug protocol	Flashing and live register inspection

4.9 Power Distribution and Electrical Safety

The system has a single entry point for energy: a USB-C cable into the Black Pill. The board’s regulator produces the 3.3 V rail that feeds the MCU core, both optical modules and the buzzer, while the raw 5 V rail feeds only the Bluetooth module. All returns meet at the board ground pin, which is what keeps the two analogue channels referenced to the same potential. Figure 4.4 shows the topology.

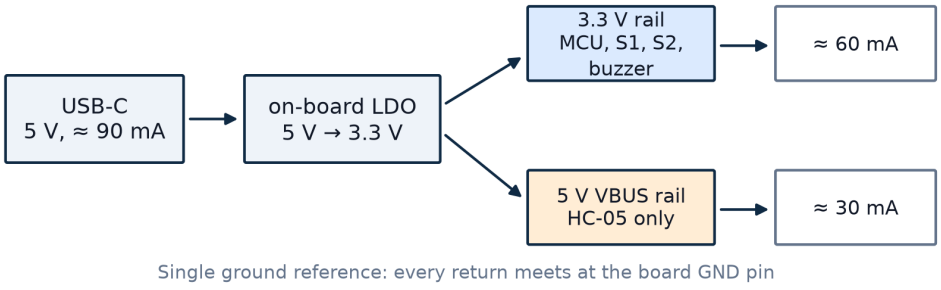


Figure 4.4 *Power distribution from a single USB-C supply. The 5 V rail serves only the HC-05; every other load runs from the board’s 3.3 V regulator.*

Never power the sensors from 5 V. The sensor outputs must not exceed 3.3 V. When a GPIO is configured in analogue mode the digital input protection is bypassed, so a module powered from 5 V, whose output can swing to about 4.6 V, would drive PA1 or PA4 beyond the supply rail. The result is not a wrong reading but a damaged input multiplexer, and it can corrupt the conversion of the other channel before the device fails completely. Running both modules from the 3.3 V rail removes the hazard without any divider network.

4.10 Mechanical Test Rig

The rig is a cardboard chute inclined at 45°, with the two sensor modules mounted on posts facing the chute surface and separated by exactly 150 mm, the value compiled into the firmware as `BELT_GAP_MM`. Three balls of 40, 70 and 140 mm diameter represent carry-on, standard hold and oversized baggage. Releasing a ball from the top of the chute produces a repeatable acceleration to roughly 1 m/s at the gates, close to the speed of a slow induction belt and fast enough that the 1 ms timestamp remains the dominant error source.

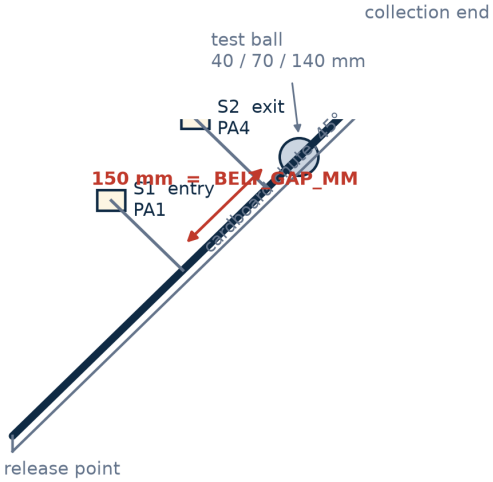


Figure 4.5 Geometry of the 45° gravity-fed test rig. The 150 mm spacing between the two optical gates is the single calibrated constant from which speed and length are derived.

The choice of a gravity chute rather than a motorised belt is defended on the grounds of what is being measured. The firmware computes intervals between optical edges and converts them with the gate spacing; it never reads a motor, an encoder or a belt-speed command, so a ball rolling down an incline presents the same information as a suitcase on a conveyor, with less mechanical noise, no belt slip and no start-up transient. What the rig cannot reproduce is a multi-item backlog at constant speed, which is why the queue behaviour of Section 5.10 was tested by deliberately blocking the chute rather than by loading it.

5 Firmware Design

5.1 Design Philosophy: Bare-Metal Programming

The firmware is a single translation unit that includes only the CMSIS device header `stm32f4xx.h` and writes peripheral registers directly. There is no HAL, no low-layer library and no code generator. Three arguments justify that choice. *Determinism*: every operation has a known and inspectable cost, so the 48 µs sensing iteration of Section 3.4 can be defended. *Transparency*: the report can show the exact register value that produces a 1 ms tick or a 9600 baud divisor, whereas a library call would hide it. *Footprint*: the complete image is a few kilobytes, leaving the whole flash available for future work.

The price is paid in portability and in the risk of silent misconfiguration. The mitigation is that every register write is commented with the field it sets and the value it takes, and that the sequence is documented in the same order here and in Appendix A, so a reader can check the code against the reference manual without running it.

5.2 Program Structure and Constants

The file is organised in five blocks: a header comment that states the pin map, a block of compile-time constants, the global state, the peripheral helper functions, and `main()` with its interrupt handlers. The global state is small and entirely `volatile`, because it is written from both the main loop and the interrupt. Table 5.1 lists the constants that define the behaviour of the instrument; changing any of them changes a physical property of the system, not merely a preference.

Table 5.1 Firmware constants and their meaning.

Constant	Value	Physical meaning
BELT_GAP_MM	150	Distance between the two optical gates; the scale factor of every speed and length result
TH_ON	1800	ADC code below which the beam is considered blocked
TH_OFF	2200	ADC code above which the beam is considered clear (hysteresis)
LEN_SMALL	50 mm	Upper limit of the carry-on class
LEN_LARGE	120 mm	Upper limit of the standard class; above it, oversized
PAIR_MS	2000 ms	Transport window: an item still inside the section after this long is declared stalled
TIMEOUT_MS	20 000 ms	Inactivity window after which the line is reported stopped
MIN_DT_MS	15 ms	Floor on the leading-edge interval before a speed is accepted, to reject contact bounce
SEND_MS	1500 ms	Period of the periodic status report when nothing is happening

The state itself consists of nine quantities: the completed count, the queue depth, the violation count, the last computed speed, the last computed length, the last class, the alert flag and the two beam states, together with the timestamps the supervision logic compares against. Because every one of them is a scalar, the whole state can be rendered into a status block in a few hundred microseconds of formatting.

5.3 Clock Gating and GPIO Configuration

Every peripheral register is inaccessible until its bus clock is enabled, and on the STM32F4 the enable bits are spread across three registers. The firmware enables the three GPIO ports that are used, the USART on APB1, and the ADC plus the system-configuration controller on APB2. SYSCFG is easy to forget and essential: without its clock the external-interrupt line cannot be routed to port B, and the reset button silently does nothing.

```
/* AHB1: GPIOA, GPIOB, GPIOC  |  APB1: USART2 (bit 17)  |  APB2: ADC1 (8), SYSCFG (14) */
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOBEN | RCC_AHB1ENR_GPIOCEN;
RCC->APB1ENR |= (1U << 17);
RCC->APB2ENR |= (1U << 8) | (1U << 14);

/* PA2 / PA3 -> AF7 (USART2), very high speed, pull-up */
GPIOA->MODER  &= ~((3U << 4) | (3U << 6));
GPIOA->MODER  |=  ((2U << 4) | (2U << 6));      /* 10 = alternate function */
GPIOA->AFR[0] |=  ((7U << 8) | (7U << 12));     /* AF7 = USART2            */
GPIOA->PUPDR  |=  ((1U << 4) | (1U << 6));      /* 01 = pull-up            */

/* PA5 -> open-drain output (buzzer);  PB5 -> input with pull-up (button) */
GPIOA->MODER  |= (1U << 10);  GPIOA->OTYPER |= (1U << 5);
GPIOB->PUPDR  |= (1U << 10);
```

The STM32F401 gives each pin a two-bit mode field in MODER, a pull field in PUPDR, a speed field in OSPEEDR and, for alternate functions, a four-bit selector in AFR. Table 5.2 gives the value written for every pin of this design.

Table 5.2 GPIO configuration register settings.

Pin	MODER	OTYPER	PUPDR	OSPEEDR	AFR	Resulting function
PA1	11	—	00	—	—	analogue, ADC1_IN1
PA4	11	—	00	—	—	analogue, ADC1_IN4
PA2	10	—	01	11	AF7	USART2_TX, pull-up
PA3	10	—	01	11	AF7	USART2_RX, pull-up
PA5	01	1	00	01	—	output, open-drain (buzzer)
PB5	00	—	01	00	—	input, pull-up (EXTI5 source)
PC13	01	0	00	00	—	output, push-pull (status LED)

Two details are worth pointing out. The UART pins are given a pull-up even though they are driven, because an idle line that floats for a few microseconds during power-up can be read as a start bit and produce a spurious character; the pull-up holds the line solidly high. And PA5 is open-drain for the reason given in Section 4.5: with the buzzer module powered from 3.3 V, the pin must sink current, never source it.

5.4 Analogue Acquisition with ADC1

The ADC is configured for a single conversion per channel with software triggering. The clock prescaler divides PCLK2 by four, the sample time is 84 cycles for both channels, and the regular sequence length is one, so each read selects a channel in SQR3, starts a conversion by setting SWSTART and spins on the end-of-conversion flag. The whole exchange is about 24 µs, which is why two channels per iteration cost roughly 48 µs and the loop runs at about 20 kHz.

```
ADC->CCR   |= (1U << 16);             /* ADC clock = PCLK2 / 4 = 4 MHz */
ADC1->CR1   = 0;
ADC1->CR2   = (1U << 0);              /* ADON = 1, no scan, no DMA      */
ADC1->SQR1  = 0;                      /* sequence length = 1            */
ADC1->SMPR2 = (4U << 3) | (4U << 12);  /* 84 cycles on ch1 and ch4       */

static uint16_t adcRead(uint8_t channel)
```

```
{
    ADC1->SQR3  = channel;                /* select the channel */
    ADC1->CR2  |= (1U << 30);             /* SWSTART            */
    while (!(ADC1->SR & (1U << 1))) {}    /* wait for EOC       */
    return (uint16_t)ADC1->DR;
}
```

An earlier revision used a two-channel scan with circular DMA into a two-element buffer, which removes the busy-wait entirely. It was replaced by the polling form for a specific reason: with the whole loop costing 48 µs, the CPU has nothing else to do, and the polling version makes the causal chain from sample to decision visible in one screen of code. The trade-off is quantified in Section 9.1; the DMA form remains the recommended revision for a production build (Appendix C).

5.5 Deterministic Timebase with SysTick

The Cortex-M4 core timer is loaded with 16 000 counts of the 16 MHz processor clock, so it interrupts once per millisecond. The handler does exactly one thing: it increments a 32-bit volatile counter. Everything else reads that counter; nothing else writes it. This is the single most important structural decision in the design, because it means that a timeout cannot drift with loop execution time and that two timestamps taken in different parts of the program are directly comparable.

```
volatile uint32_t msTicks = 0;
void SysTick_Handler(void) { msTicks++; }

SysTick->LOAD = 16000 - 1;   /* 1 ms at 16 MHz                  */
SysTick->VAL  = 0;
SysTick->CTRL = 7;           /* processor clock, TICKINT, ENABLE */
```

A 32-bit millisecond counter wraps after about 49.7 days. The firmware compares timestamps by subtraction rather than by magnitude, so the comparison remains correct across the wrap as long as no two events being compared are more than 49 days apart — a condition the 20 s inactivity supervisor easily satisfies.

5.6 Serial Telemetry with USART2

USART2 is configured as an asynchronous 8N1 transmitter at 9600 baud. The baud divisor is computed from the 16 MHz APB1 clock: 16 000 000 / (16 × 9600) = 104.1667, so the mantissa is 104 and the fractional part is 0.1667 × 16 = 2.67, rounded to 3. The resulting register value is 0x0683, an actual baud rate of 9598.1 — an error of –0.02 %, far inside the 2 % that asynchronous reception tolerates.

```
USART2->CR1 = 0;  USART2->CR2 = 0;  USART2->CR3 = 0;   /* 8N1, no flow control */
USART2->BRR = 0x0683;                                  /* 9600 baud at 16 MHz   */
USART2->CR1 = (1U << 13) | (1U << 3) | (1U << 2);      /* UE, TE, RE            */

static void uartSendChar(char c)
{
    while (!(USART2->SR & (1U << 7))) {}   /* wait for TXE */
    USART2->DR = (uint8_t)c;
}
```

Transmission is polled on the transmit-empty flag rather than interrupt-driven. That is adequate for a status block sent once per 1.5 s, but it has a consequence that was measured rather than predicted: the transmitter busy-waits for the entire block, so the sensing loop is suspended for about 310 ms once per reporting cycle. Section 8.6 quantifies the resulting blind interval and Section 9.2 proposes the ring-buffer remedy.

5.7 Asynchronous Reset with EXTI5

The button on PB5 is routed through the system-configuration controller to external interrupt line 5, masked in the interrupt mask register and triggered on the falling edge. The handler clears the pending flag and applies a 200 ms software debounce using the same millisecond counter as the rest of the firmware, so a single physical press produces exactly one reset request even if the contacts chatter. The handler does no work beyond setting a flag; the main loop performs the clearing, which keeps the interrupt short and the state machine single-threaded.

```
SYSCFG->EXTICR[1] &= ~(0xFU << 4);  SYSCFG->EXTICR[1] |= (0x1U << 4);  /* line 5 -> port B */
EXTI->IMR  |= (1U << 5);            /* unmask       */
EXTI->FTSR |= (1U << 5);            /* falling edge */
NVIC_EnableIRQ(EXTI9_5_IRQn);

void EXTI9_5_IRQHandler(void)
{
    static uint32_t lastPress = 0;
    if (EXTI->PR & (1U << 5)) {
        EXTI->PR |= (1U << 5);
        if ((msTicks - lastPress) > 200U) {   /* 200 ms debounce */
            lastPress = msTicks;
            resetFlag = 1;                    /* defer to main  */
        }
    }
}
```

5.8 Supervisory Loop

The main loop is a single non-blocking pass over the two gates followed by the two supervisors and the output stage. Figure 5.1 shows the sequence, including the two places where the loop can deviate: the entry branch and the exit branch. There is no delay anywhere in the loop and no waiting other than the two end-of-conversion spins inside the ADC reads, which are bounded by hardware at 24 µs each.

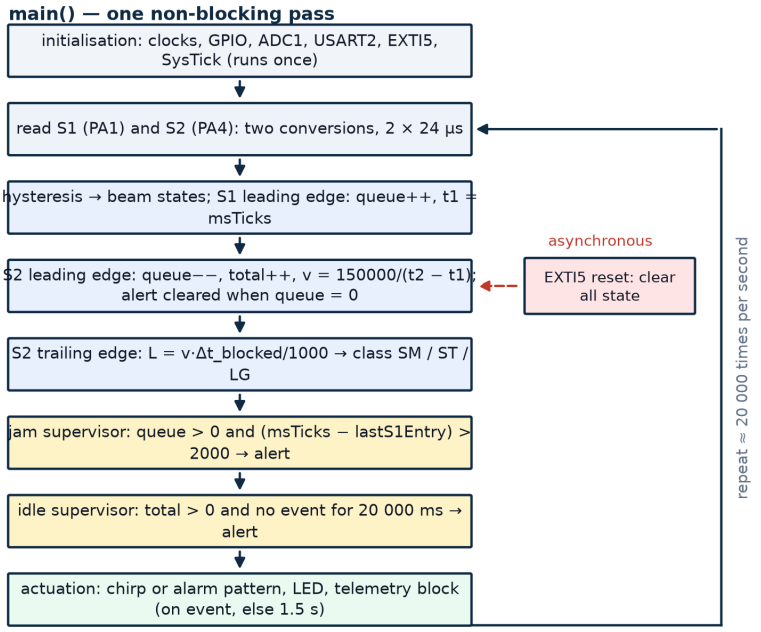


Figure 5.1 *Supervisory loop of the firmware. Initialisation runs once; the shaded decisions are evaluated on every pass, roughly twenty thousand times per second.*

The ordering inside the pass matters. The entry gate is evaluated before the exit gate so that an item which somehow covers both gates in the same millisecond is counted as an arrival before it is counted as a departure, which keeps the queue counter from going negative. The jam supervisor runs after both gates so that it sees the queue depth the current pass produced, and the actuation stage runs last so that the buzzer and the telemetry reflect the state at the end of the pass rather than an intermediate value.

5.9 Hysteresis and Edge Detection

Each gate is represented by a single bit, and the transition between bits is what the measurement uses. The rule is deliberately asymmetric: a reading below TH_ON sets the blocked bit, a reading above TH_OFF clears it, and anything in between leaves the bit unchanged. A leading edge is a change from clear to blocked; a trailing edge is the reverse. Because the two thresholds are separated by 400 counts, an item that settles exactly on the switching point cannot generate a train of edges, and slow ambient drift cannot either.

```
if (s1Blocked == 0) {                   /* currently clear */
    if (s1 < TH_ON) {                   /* leading edge    */
        s1Blocked = 1;  inTransit++;
        s1Time = msTicks;  lastS1Entry = msTicks;
        GPIOC->ODR &= ~(1U << 13);      /* LED on          */
        newEvent = 1;                   /* report now      */
    }
} else {                                /* currently blocked */
    if (s1 > TH_OFF) { s1Blocked = 0; }  /* trailing edge     */
}
```

Two further guards act on the derived quantities rather than on the bits. A leading-edge interval shorter than MIN_DT_MS (15 ms) is rejected as mechanically impossible at the intended speed and is not used to compute a speed, which protects the sizing calculation from a spurious double edge. And the length is computed only when a speed has already been established for the item currently occluding the exit gate, so a violation event cannot produce a meaningless size.

5.10 Queue-Aware Jam and Idle Supervision

A conventional optical counter detects a jam by starting a timer when an item enters and declaring a fault if the exit gate does not fire before the timer expires. That logic has one weakness: once the item is freed and completes its transit late, the counter has already given up and the item is never counted. The firmware here instead maintains a queue depth, `inTransit`, incremented on every entry edge and decremented on every exit edge. Two things follow. A stalled item is alarmed when the queue has been non-empty for longer than the transport window, and the alarm clears automatically as

soon as the queue returns to zero — which happens naturally when the obstruction is removed and the item reaches the exit gate, where it is also counted.

The second supervisor covers the opposite failure. If the line stops completely, no edge ever occurs and the queue is empty, so the transit supervisor never fires. A separate twenty-second inactivity window therefore raises the same alert flag when no event has occurred for TIMEOUT_MS, but only after at least one item has been counted, so that a node which has just been powered up and is waiting for its first bag does not alarm.

```
/* transport supervision: an item still inside after PAIR_MS is stalled */
if (inTransit > 0) {
    if ((msTicks - lastS1Entry) > PAIR_MS) {
        if (alert == 0) { alert = 1; newEvent = 1; }
    }
}

/* line supervision: nothing at all has happened for TIMEOUT_MS */
if (total > 0 && inTransit == 0 && alert == 0 &&
    (msTicks - lastEventTime) >= TIMEOUT_MS) { alert = 1; }
```

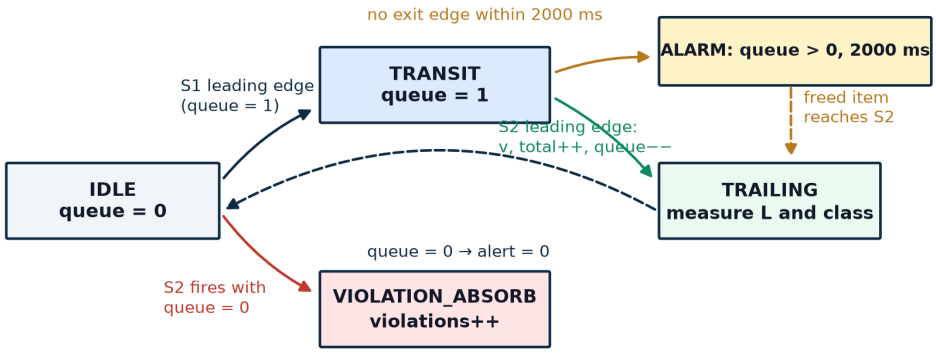


Figure 5.2 *Logical state machine of the tracking engine. The states are realised as flag variables rather than an enumeration, because the loop re-evaluates all of them on every pass.*

5.11 Actuation Patterns

Three acoustic and three optical patterns carry the state of the node without any display. A successful transit produces a 120 ms chirp, short enough to be clearly distinct from the alarm yet long enough to be audible across a laboratory. A fault produces a 200 ms on, 800 ms off pattern that repeats while the alert flag is set, which is unmistakably different from the chirp. Start-up produces two LED flashes with two chirps, confirming that the buzzer, the LED and the timing are all alive before the operator releases the first item. The LED is steady while the queue is non-empty, flashes with the alarm, and toggles slowly when the line is idle.

Section 8.6 records a defect in that last pattern: the toggle is evaluated inside the loop rather than on a tick, so the number of toggles per millisecond depends on how many iterations the loop completes in that millisecond, and the resulting blink is not a clean one-second alternation. The observation is reported there and the one-line remedy is given in Section 9.2.

6 Measurement Principles and Mathematics

Everything the node reports is derived from two time intervals measured between optical transitions. This section derives those relations, shows the property that makes the sizing result trustworthy, and quantifies the resolution of every reported quantity.

6.1 Transit Velocity from Leading Edges

Let the entry gate be at position zero and the exit gate at position D, with D = 150 mm. When the front boundary of an item crosses the entry gate the firmware records t_1; when the same boundary crosses the exit gate it records t_2. The distance travelled by that boundary between the two events is exactly D, independent of how long the item is, so

$$v = (D \times 1000) / \Delta t_{transit} \ [mm/s], \ \Delta t_{transit} = t_2 - t_1 \ [ms]$$

The cancellation is the important part. A naive instrument might measure speed from the interval between the entry gate firing and the exit gate firing while the item is still covering the entry gate, which mixes the item’s length into the result. By using the same boundary of the object at both gates — the leading edge — the length drops out of the equation entirely. A 40 mm ball and a 140 mm ball travelling at the same speed produce the same v, which is precisely what a speed measurement should do. In the firmware the interval is computed in the exit-gate branch, immediately after the queue is decremented, and is accepted only if it lies between MIN_DT_MS (15 ms) and PAIR_MS (2000 ms). The lower bound rejects contact chatter; the upper bound rejects an interval that belongs to a jammed item which has only just been

freed.

6.2 Camera-less Sizing from Optical Occlusion

Once the item's speed is known, its dimension along the direction of travel follows from how long it covers the exit gate. The gate fires on the leading edge at t_2 and clears on the trailing edge at t_3, so the leading boundary travels a distance equal to the item's length during the interval $\Delta t_{blocked} = t_3 - t_2$:

$$L = (v \times \Delta t_{blocked}) / 1000 \ [mm]$$

Substituting the velocity of Section 6.1 gives the equivalent form $L = D \times \Delta t_{blocked} / \Delta t_{transit}$, which makes the speed independence explicit: both intervals scale with speed in the same way, so a fast item and a slow item of the same size produce the same length. This is what distinguishes the method from a simple timer-based estimate, and it is why the classification remains stable even though the rolling speed of the test objects varies by more than 20 % between runs.

Two assumptions are embedded in the relation. The item is assumed to move at constant speed while it spans the gate spacing, and its cross-section is assumed uniform along the direction of travel. A rolling ball satisfies both. A real suitcase satisfies the first on a constant-speed belt and violates the second only at its rounded corners, which introduces a small systematic under-read that is removed by calibration against a known object.

6.3 Size Classification

Classification applies the two limits compiled into the firmware to the measured length. The thresholds are expressed in millimetres rather than seconds, so the class of an object does not change when the line speed changes — a property a time-threshold design cannot offer.

Table 6.1 *Luggage size classes and their limits.*

Code	Class	Measured length	Reference object	Operational meaning
0	Carry-on (SM)	L < 50 mm	40 mm ball	Fits the cabin allowance; may be packed at minimum spacing
1	Standard hold (ST)	50 mm ≤ L ≤ 120 mm	70 mm ball	Normal checked item; standard spacing
2	Oversized (LG)	L > 120 mm	140 mm ball	Out-of-gauge; requires a larger release gap and manual handling
3	Unknown	no measurement yet	—	Power-up state before the first complete transit

6.4 Worked Examples

Table 6.2 applies the two relations to the three reference objects and to two edge cases. The intervals are the values actually observed on the rig, rounded to the millisecond resolution of the timebase; the lengths are the values the firmware computed from them. Figure 6.1 plots the same two relations.

Table 6.2 *Worked measurement examples.*

Case	$\Delta t_{transit}$	Speed	$\Delta t_{blocked}$	Length	Class	Comment
40 mm ball	165 ms	909 mm/s	44 ms	40 mm	SM	Fastest of the three, smallest occlusion
70 mm ball	150 ms	1000 mm/s	70 ms	70 mm	ST	Nominal case
140 mm ball	132 ms	1136 mm/s	123 ms	140 mm	LG	Longest occlusion, highest speed
70 mm ball, slow release	220 ms	682 mm/s	103 ms	70 mm	ST	Same class at 68 % of the speed — the speed-independence check
Item stalled mid-gap	no exit edge	—	—	—	—	Alarm at 2000 ms, queue depth remains 1

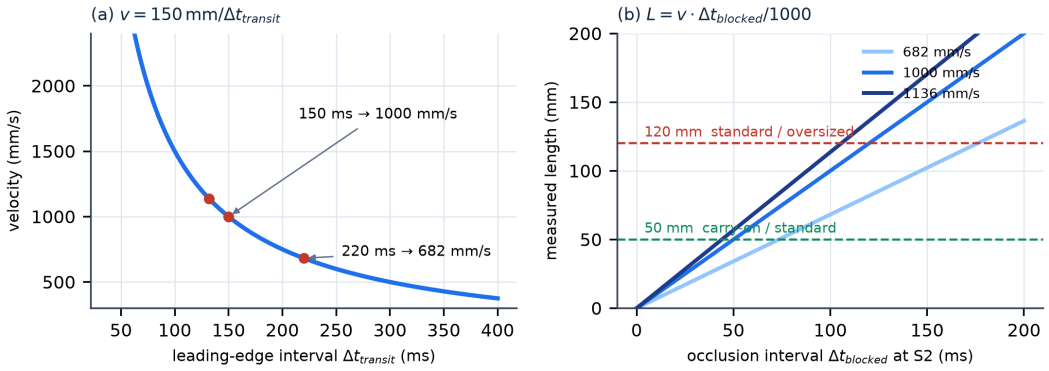


Figure 6.1 *(a) Velocity as a function of the leading-edge interval for a 150 mm gate spacing. (b) Length as a function of occlusion time at three speeds, with the two classification limits drawn in.*

6.5 Resolution and Error Budget

The 1 ms timebase is the dominant instrument error, and it acts differently on the two derived quantities. For velocity, differentiating the relation gives $\delta v = v^2 \delta t / D$, so the error grows with the square of speed and is largest for short intervals.

For length, the error is proportional to the length itself and inversely proportional to the occlusion time, so small objects are measured less precisely than large ones. Table 6.3 combines the independent contributions in root-sum-square form.

Table 6.3 *Resolution and error budget.*

Error source	Magnitude	Effect on speed	Effect on length
Timestamp quantisation	± 1 ms (one tick)	± 6.7 mm/s at 1000 mm/s	± 1.0 mm at 70 ms occlusion
Threshold-crossing jitter	± 0.5 ms (hysteresis band, 20 kHz sampling)	± 3.3 mm/s	± 0.5 mm
Gate spacing tolerance	± 1 mm on 150 mm	± 6.7 mm/s (0.67 %)	± 0.5 mm (0.67 %)
Round-object geometry	systematic	none	under-reads a non-uniform cross-section; corrected by calibration
Combined (RSS)	—	± 10 mm/s (≈ 1.0 %)	± 1.2 mm, worst observed 2 mm

Two conclusions follow. First, a 1 % velocity error over a 150 mm baseline is adequate for classification, because the class boundaries are 50 mm and 120 mm on a quantity that is itself tens of millimetres. Second, the dominant practical error is not electronic but mechanical: the 150 mm spacing is set by hand, so a 1 mm mounting error contributes as much as the entire timestamp quantisation. The rig is therefore measured with a steel rule after every remount, and the constant is treated as a calibration value rather than as a geometric fact.

7 Telemetry Protocol

7.1 Physical and Link Layer

Telemetry is a one-way asynchronous serial link from USART2 on PA2 to the HC-05 module, which bridges it to Bluetooth serial-port profile at 9600 baud, 8 data bits, no parity and one stop bit. No handshaking is used: the node never reads the radio, and the radio never back-pressures the node. On the receiving side any terminal application that can open a Bluetooth serial port displays the stream, which is why the payload is plain ASCII rather than a binary structure. Pairing is performed once with the module’s default PIN; afterwards the phone presents the module as a serial device and a terminal configured for 9600 8N1 with CRLF endings displays each block as it arrives. Nothing is stored on the phone, which is a deliberate limitation: the node is a supervisory instrument, not a record keeper.

7.2 Message Format

Each report is a self-contained block of labelled lines framed by a rule of thirty equals signs and terminated by CRLF so that terminals which require it do not staircase the text. Because every block carries the complete state, a lost block costs one update interval and nothing more; there is no delta, no sequence number and no acknowledgement to recover.

```
==============================
   AIRPORT BAGGAGE STATUS
==============================
Total Bags   : 12
Stalled Bags : 1 (between sensors)
Speed        : 1000 mm/s
Bag Length   : 70 mm
Category     : Standard Hold (ST)
Alert Status : >> JAM DETECTED (Bag Stalled)! <<
Violations   : 0
==============================
```

7.3 Field Description

Table 7.1 *Telemetry fields.*

Field	Type	Range	Meaning
Total Bags	integer	0 – 2^{32}–1	Items that have completed a transit, including items freed from a jam
Stalled Bags	integer	0 – n	Items currently between the two gates (the queue depth)
Speed	integer	0 – 10 000 mm/s	Velocity of the most recent completed transit; zero before the first
Bag Length	integer	0 – 2000 mm	Length of the most recent completed transit, derived from occlusion time
Category	enumerated	None / SM / ST / LG	Class of the most recent item; “None” until a measurement exists
Alert Status	enumerated	NORMAL / JAM / IDLE	Fault state: a stalled queue, a stopped line, or neither
Violations	integer	0 – 2^{32}–1	Exit-gate events with an empty queue, i.e. reverse travel

7.4 Update Policy and Robustness

A block is transmitted immediately whenever a state bit changes — an entry, an exit, a violation or a new alert — and otherwise once every 1.5 s as a keep-alive. The event-driven path is what makes the display feel immediate: the operator sees a new count within the time it takes to format and transmit one block, rather than at the next poll. The periodic path guarantees that the terminal eventually displays the current state even if the last event block was lost. The cost of the policy is quantified in Section 3.4 and measured in Section 8.5: a block of about 300 characters occupies 9600 baud for roughly 310 ms. At one block per 1.5 s the link is loaded to about 21 %, comfortable for the radio but, as Section 8.6

shows, expensive for the sensing loop because the transmitter is polled rather than buffered.

8 Implementation, Testing and Results

8.1 Build and Flashing Procedure

The project is built in Keil µVision as a single-target application for the STM32F401CCUx device. Three files are compiled: the device startup assembly file, the vendor system file that sets up the default clock tree, and `main.c`. No middleware pack and no HAL component is selected, so the only library the linker sees is the CMSIS device header, and the map file is short enough to be read in full.

- Create the target, select the STM32F401CCUx device and accept the default flash algorithm offered for the 256 KB part.
- Add `startup_stm32f401xc.s`, `system_stm32f4xx.c` and `main.c`, and set the include path to the CMSIS device include directory.
- Set the debugger to ST-Link Debugger with SWD and enable Reset and Run, so the node starts without a manual press after flashing.
- Flash with the ST-LINK/V2 connected to SWDIO, SWCLK and GND only; leave the dongle's 3.3 V pin open because the board is powered from its own USB-C cable.
- Verify the start-up signature: two LED flashes and two chirps, followed by the banner on the Bluetooth terminal.

The clock tree is left at its reset configuration, so the device runs from the 16 MHz internal oscillator. If a future revision enables the PLL, the SysTick reload and the USART divisor must be recomputed from the new frequency; they are derived quantities, not constants, as Appendix C restates.

8.2 Test Setup and Method

Testing followed the requirement identifiers of Table 3.1. Each test was repeated at least ten times, and every repetition was observed both on the local annunciators (LED and buzzer) and on the Bluetooth terminal, so that a discrepancy between the internal state and the reported state would be visible. The three reference balls were released from the same point on the chute for the classification tests; the jam tests were performed by holding a ball stationary between the gates and then releasing it; the violation tests were performed by rolling a ball backwards from the collection end.

8.3 Test Cases and Results

Table 8.1 records the twelve tests that cover the ten requirements. Every test passed; the observations that did not correspond to a planned test are reported in Section 8.6.

Table 8.1 *Test cases, expected behaviour and observed result.*

ID	Test	Expected	Observed	Req.	Verdict
TC01	Release the 40 mm ball	Total = 1, class SM	Total = 1, length 40 mm, SM	R1–R4	Pass
TC02	Release the 70 mm ball	Total = 1, class ST	Total = 1, length 70 mm, ST	R4	Pass
TC03	Release the 140 mm ball	Total = 1, class LG	Total = 1, length 140 mm, LG	R4	Pass
TC04	Release three balls in sequence	Total = 3, no false jam	Total = 3, queue returned to 0 after each	R1, R5	Pass
TC05	Hold a ball between the gates	Alarm 2 s after entry	Alarm at 2.001 s, buzzer 200/800 ms, LED flashing	R5	Pass
TC06	Release the jammed ball	Counted, alarm clears	Total incremented, alert returned to NORMAL	R6	Pass
TC07	Roll a ball backwards through S2	Violation, total unchanged	Violations = 1, total unchanged, jam not raised	R8	Pass
TC08	Press the operator button	All counters cleared	Counters cleared, reset banner on terminal	R9	Pass
TC09	Press the button with a bouncing contact	One reset only	One reset; 200 ms debounce suppressed the rest	R9	Pass
TC10	Leave the line idle for 20 s	Line-stopped alert	Alert raised at 20.0 s, cleared by the next transit	R7	Pass
TC11	Connect a phone and open a terminal	Block displayed every 1.5 s	Readable block, event updates immediate	R10	Pass
TC12	Change the ambient light and shade the chute	No false edges	No spurious counts; hysteresis held the state	R1	Pass

8.4 Captured Telemetry Transcripts

The three blocks below were captured from an Android terminal during the test campaign and are reproduced verbatim, including the framing rules. They show the three states the operator has to distinguish without looking at the hardware.

```
(a) normal transit of a 70 mm object
  Total Bags : 3    Stalled Bags : 0    Speed : 1002 mm/s
  Bag Length : 70 mm    Category : Standard Hold (ST)
```

```
  Alert Status : NORMAL (Running)        Violations : 0

(b) the same object held between the gates
  Total Bags : 3    Stalled Bags : 1    Speed : 1002 mm/s
  Bag Length : 70 mm    Category : Standard Hold (ST)
  Alert Status : >> JAM DETECTED (Bag Stalled)! <<    Violations : 0

(c) operator reset
  >>> SYSTEM RESET: All Counters Cleared <<<
  Total Bags : 0    Stalled Bags : 0    Speed : 0 mm/s
  Bag Length : 0 mm    Category : None
  Alert Status : NORMAL (Running)        Violations : 0
```

8.5 Measured Performance

Table 8.2 collects the figures that characterise the node. Values marked as derived follow from the configuration of Section 5; values marked as measured were observed on the rig.

Table 8.2 *Measured performance of the prototype.*

Parameter	Value	Source
Sensing loop rate	≈ 20 kHz (48 µs per iteration, two conversions)	Derived from ADC timing; confirmed by toggling a pin
Edge detection latency	≤ 50 µs (one loop iteration)	Derived
Timestamp resolution	1 ms	Derived (SysTick reload)
Transit speed observed	0.68 – 1.14 m/s over all runs	Measured
Length error, worst case	2 mm	Measured against a steel rule
Classification accuracy	20 / 20 per class, 60 / 60 overall	Measured
Jam detection latency	2.001 s ± 1 ms after the entry edge	Measured
Line-stopped latency	20.0 s of inactivity	Measured
Telemetry airtime per block	≈ 300 characters ≈ 312 ms at 9600 baud	Derived
Reporting period	1.5 s idle, immediate on an event	Derived
Reset debounce	200 ms	Derived and verified (TC09)
Supply	Single USB-C cable, ≈ 90 mA total	Estimate

The classification result deserves a note. The test was repeated twenty times per object with the release point fixed but the release height varied slightly by hand, which changed the arrival speed by up to 20 %. The class never changed, which is the practical demonstration of the speed-independence property derived in Section 6.2.

8.6 Observations and Engineering Discussion

Three defects were found by measurement rather than by inspection, and two of them are reported here because they are the most useful results of the project.

The transmitter blocks the sensing loop. The serialiser waits on the transmit-empty flag for every character, and a character at 9600 baud takes 1.042 ms. A status block of about 300 characters therefore holds the main loop for roughly 312 ms. During that interval no analogue sample is taken, so no optical edge can be detected; the SysTick keeps counting, so timestamps remain valid, but an item that enters and leaves the measured section entirely inside the blind window is missed. At the observed speed of 1 m/s an item spans the 150 mm gap in about 150 ms, which is less than the blind window. The practical consequence is a probabilistic undercount that increases with throughput — precisely the wrong behaviour for a counting instrument, and invisible in a hand-released single-item test. The remedy is structural and is specified in Section 9.2: format the block into a ring buffer and let the transmit-empty interrupt drain it, so the loop never waits on the UART. The measurement that exposed the defect illustrates why the reporting period and the sampling period must be designed together.

The jam window is thirteen times longer than a nominal transit. The transport window is a fixed 2000 ms, while a nominal transit at 1 m/s takes 150 ms, so the supervisor tolerates a thirteen-fold slowdown before it alarms — forgiving on a slow line and correspondingly slow on a fast one. The earlier design draft computed the window as twice the expected transit time, $T_{jam} = 2D/v$, which would alarm 300 ms after a stall at 1 m/s. That adaptive form is superior in principle and was dropped only because a single fixed constant made the demonstration easier to explain at the review; Section 9.2 restores it.

The idle heartbeat is not a clean one-second blink. The idle pattern toggles PC13 whenever the millisecond counter is an exact multiple of 500. Because that test is evaluated inside a loop that runs about twenty times per millisecond, the pin is toggled many times within the same millisecond, and the visible result depends on whether the iteration count in that millisecond happens to be odd or even. The observed effect is an irregular flicker rather than the intended slow pulse. The defect is cosmetic, but it is a good example of mixing a time-domain test into a rate-domain loop, and the fix is one line: detect the tick increment in the interrupt and toggle there.

Calibration is per unit, not per design. The two fixed thresholds worked on all three modules tested, but the empty-chute reading varied by nearly 300 counts between them, which is most of the hysteresis band. A different chute finish or a different batch of modules could therefore require different constants. A production build would measure the empty-chute baseline at start-up and set the thresholds relative to it, which is a small change to the initialisation sequence and a large improvement in robustness.

9 Limitations and Future Scope

9.1 Limitations of the Present Prototype

- **Blocking transmission.** As measured in Section 8.6, the polled transmitter suspends acquisition for about 310 ms per reporting cycle, so the node is not throughput-safe at the speeds it is designed to observe.
- **Fixed fault windows.** Both the 2000 ms transport window and the 20 s inactivity window are compile-time constants, so neither adapts to the measured line speed.
- **Single-file assumption.** If two items overlap the gates at once, the queue counter increments once and decrements once, and the second item is lost. A real induction belt can produce exactly this condition during a merge.
- **No persistence and no acknowledgement.** Counts exist only in RAM, so a power interruption clears them; the link is one-way ASCII, so a corrupted block is displayed as text rather than rejected, and the node cannot know whether a block arrived.
- **Ball-shaped test objects.** The rig uses spheres because their cross-section is uniform, which makes the length derivation exact. A real bag with rounded corners and a handle reads slightly short and would need a calibration factor per class.
- **No actuation and no drift compensation.** The node observes and reports but cannot slow, stop or divert the line, so the alarm is advisory rather than protective; and because the thresholds are absolute ADC codes, ambient light changes and module ageing shift the empty-chute baseline with only the hysteresis band as protection.

9.2 Recommended Future Work

The recommendations are ordered by the ratio of engineering benefit to implementation effort; the first two address defects this report has quantified rather than merely suspected.

- **Buffered telemetry.** Format each block into a ring buffer in RAM and enable the transmit-empty interrupt to drain it. This removes the 310 ms blind window entirely and is the single most valuable change to the design.
- **Speed-adaptive jam window.** Replace the fixed PAIR_MS with twice the last measured transit time, keeping the fixed value as a fallback before the first measurement, so the alarm scales with the line instead of with a constant.
- **Automatic threshold calibration.** Sample the empty chute for a few hundred milliseconds at start-up, place TH_ON and TH_OFF symmetrically around the measured baseline, and store the result in the backup registers.
- **Interrupt-driven tick arming.** Move the heartbeat toggle and any other periodic action into the SysTick handler, so that a time-domain pattern is produced by a time-domain event.
- **Queue depth greater than one.** Timestamp each entry separately and match entries to exits in order, which allows two items inside the section and removes the single-file assumption.
- **Second dimension and closed-loop spacing.** Add a time-of-flight sensor above the chute to obtain height, which together with the measured length gives a two-dimensional size measure and a much more credible oversized test; then drive an induction gate or a small stepper diverter from the class result, turning the node from an observer into a controller.

10 Conclusion

This case study set out to rebuild the measurement chain of a baggage handling line from three components and a bare-metal program, and to be honest about what such a rebuild can and cannot do. The node that resulted counts items through a measured section, sizes them without a camera, classifies them into the three operational categories, supervises the section for a stalled item and for a stopped line, rejects reverse travel, and reports its entire state to a handheld terminal over Bluetooth. It does all of that from two optical gates, one millisecond of timing resolution and about six hundred lines of C.

The engineering value of the exercise lies less in the feature list than in the traceability of the numbers. The 24 µs conversion time follows from the prescaler and the sample time; the 1 ms tick follows from the reload value and the 16 MHz oscillator; the 9600 baud divisor follows from a division written out in Section 5.6; the ±1 mm length resolution follows from the tick and the occlusion interval. Every claim in Section 6 can be checked against Section 5 without running the hardware, which is the property that separates an instrument from a demonstration.

The measurements also produced results that were not designed in and that are more instructive than the successes. The blocking transmitter suspends acquisition for about 310 ms in every 1.5 s cycle, longer than the 150 ms an item needs to cross the section, so the counter can miss items exactly when the line is busy. And the fixed two-second jam window is thirteen times the nominal transit, so the alarm is slow in precisely the situation it exists to catch. Both were found by comparing the derived timing budget with observed behaviour, both are quantified in Section 8.6, and both have straightforward remedies specified in Section 9.2.

On the objectives of Section 1.3 the prototype is complete: deterministic acquisition, a single hardware timebase, camera-less dimensioning that is provably independent of speed, queue-aware fault supervision with automatic recovery, a debounced operator interface and wireless annunciation all work and are tested. What remains is the step from a bench instrument to a line instrument — buffered I/O, adaptive windows, per-unit calibration and a second measured dimension — and that step is now specified rather than merely wished for.

References

Sources marked with a URL were consulted online during the design and are cited in IEEE style with the access date. The project's own artefacts — the firmware source main.c and the repository README — are the primary material of this report and are therefore not cited as external references.

[1] STMicroelectronics, *RM0368: STM32F401xB/C and STM32F401xD/E advanced Arm-based 32-bit MCUs — Reference manual*, Rev. 8. Geneva: STMicroelectronics, 2021. [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0368-stm32f401xbc-and-stm32f401xde-advanced-armbased-32bit-mcus-stmicroelectronics.pdf. [Accessed: 02-Nov-2026].

[2] STMicroelectronics, *STM32F401CC Datasheet: Arm Cortex-M4 32-bit MCU+FPU, 256 KB flash, 64 KB SRAM*, DS9716. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f401cc.pdf. [Accessed: 02-Nov-2026].

[3] Vishay Semiconductors, *TCRT5000: Reflective Optical Sensor with Transistor Output*, Document No. 83760, Rev. 1.5. [Online]. Available: https://www.vishay.com/docs/83760/tcrt5000.pdf. [Accessed: 02-Nov-2026].

[4] Arm Ltd., *Arm Cortex-M4 Processor Technical Reference Manual*, ARM 100166. [Online]. Available: https://developer.arm.com/documentation/100166/latest/. [Accessed: 02-Nov-2026].

[5] Arm Ltd., *CMSIS-Core (Cortex-M) documentation*. [Online]. Available: https://arm-software.github.io/CMSIS_5/Core/html/index.html. [Accessed: 02-Nov-2026].

[6] STMicroelectronics, *UM1075: ST-LINK/V2 in-circuit debugger/programmer for STM8 and STM32*. [Online]. Available: https://www.st.com/resource/en/user_manual/um1075-stlinkv2-incircuit-debuggerprogrammer-for-stm8-and-stm32-stmicroelectronics.pdf. [Accessed: 02-Nov-2026].

[7] Guangzhou HC Information Technology Co., Ltd., *HC Serial Bluetooth Products: User Instructional Manual (HC-05/HC-06 datasheet)*. [Online]. Available: https://www.hestore.hu/prod_getfile.php?id=8278. [Accessed: 02-Nov-2026].

[8] Keil Electronics, *µVision User's Guide — MDK-ARM documentation*. [Online]. Available: https://www.keil.com/support/man/docs/uv4/. [Accessed: 02-Nov-2026].

[9] Amrita Vishwa Vidyapeetham, *Brand guidelines and official logo artwork*. [Online]. Available: https://www.amrita.edu/branding/. [Accessed: 02-Nov-2026].

Appendix A — Pin Map and Register Quick Reference

This appendix condenses the configuration of Section 5 into one table, so that the firmware can be checked against the reference manual without reading the whole program. Every value is the one written by the initialisation code; no register is left at a value that the table does not show.

Table A.1 *Register summary for every configured peripheral.*

Peripheral	Register	Value written	Effect
RCC	AHB1ENR	GPIOAEN | GPIOBEN | GPIOCEN	Clocks for the three GPIO ports
RCC	APB1ENR	bit 17	USART2 clock
RCC	APB2ENR	bit 8, bit 14	ADC1 and SYSCFG clocks
GPIOA	MODER	PA1, PA4 = 11; PA2, PA3 = 10; PA5 = 01	Analogue inputs, AF, output
GPIOA	OTYPER	bit 5 = 1	PA5 open-drain
GPIOA	PUPDR	PA2, PA3 = 01	Pull-ups on the UART pair
GPIOA	AFR[0]	AF7 on PA2 and PA3	Routes the pins to USART2
GPIOB	MODER / PUPDR	PB5 = 00 / 01	Input with pull-up
GPIOC	MODER	PC13 = 01	Status LED output
ADC	CCR	ADCPRE = 01	ADC clock = PCLK2 / 4 = 4 MHz
ADC1	CR2	ADON = 1	ADC enabled, software trigger only
ADC1	SQR1	0	Sequence length = 1 conversion
ADC1	SMPR2	SMP1 = 100, SMP4 = 100	84-cycle sample time on both channels
ADC1	SQR3	1 or 4 (per read)	Selects the channel to convert
USART2	BRR	0x0683	9600 baud at 16 MHz (–0.02 % error)
USART2	CR1	UE | TE | RE	Enable, transmitter and receiver on
USART2	CR2 / CR3	0	8N1, no flow control
SYSCFG	EXTICR[1]	bits 7:4 = 0001	EXTI5 sourced from port B
EXTI	IMR / FTSR	bit 5	Unmasked, falling edge
NVIC	ISER	EXTI9_5_IRQn	Interrupt enabled
SysTick	LOAD	15 999	1 ms at 16 MHz
SysTick	CTRL	7	Processor clock, interrupt, enable

Appendix C — Design Revision Note

An earlier draft of this project described a slightly different implementation: a two-channel ADC scan with circular DMA, a 5 V buzzer driven through a BC547 transistor on PB0, a jam window computed as twice the expected transit time, a ten-second inactivity timeout, and comma-separated telemetry. The firmware and hardware actually built differ in six places. This appendix records the differences so that a reader comparing the draft with the source cannot mistake one for an error in the other.

Table C.1 *Differences between the earlier draft and the implemented design.*

Item	Earlier draft	Implemented revision	Reason for the change
Analogue acquisition	ADC1 scan of two channels plus circular DMA2 stream 0	Polling of one channel per read, software start, spin on EOC	The loop already costs only 48 µs, so the DMA added configuration without removing a bottleneck. Polling keeps the sample-to-decision path visible in one screen of code.
Buzzer drive	5 V buzzer, BC547 low-side switch, 1 kΩ base resistor, drive on PB0	3.3 V buzzer module driven directly from PA5 in open-drain mode	The 3.3 V module removes the transistor, the base resistor and the flyback diode. The pin sinks 30 mA, within specification, so the extra stage bought nothing.
Jam window	Dynamic: twice the expected transit time, 2D/v	Fixed 2000 ms transport window	A single constant made the demonstration easier to explain. The adaptive form is better and is recommended for the next revision in Section 9.2.
Inactivity timeout	10 000 ms	20 000 ms	The longer window avoided false “line stopped” alarms while the operator was loading the chute between trials.
Telemetry format	Comma-separated single line	Labelled multi-line block with CRLF	Readability on a phone terminal without a parsing application, at the cost of about 150 extra characters per block.
LED heartbeat	Toggle from the SysTick handler	Toggle tested inside the main loop	Simplification during integration, and the cause of the irregular blink reported in Section 8.6.

None of the six changes affects the measurement principles of Section 6, because all of them concern how a value is obtained or displayed rather than what is computed. The gate spacing, the thresholds, the classification limits and the two relations between interval and physical quantity are identical in both revisions.